Name	AYUSHI JAPSARE
UID no.	2023301006
EXP	6
Title:	Implementing Pretty Good Privacy (PGP)

CSS	
AIM:	To implement Pretty Good Privacy (PGP) operations using GnuPG (GPG) including key generation, encryption, decryption, digital signatures, and key management.
PROBLEM STATEMENT:	1. Generate a PGP key pair (public/private). 2. Encrypt and decrypt files and messages using PGP. 3. Create and verify digital signatures. 4. Export/import keys and demonstrate basic key management (trust, revocation certificate). 5. Demonstrate use of PGP in a simple email/file-workflow Pretty Good Privacy (PGP) provides confidentiality, integrity, authentication, and non-repudiation using a combination of asymmetric (public-key) cryptography and symmetric encryption. Typical operations: - Encryption: Sender uses recipient's public key to encrypt; recipient uses their private key to decrypt. - Signing: Sender uses own private key to sign; anyone with sender's public key can verify authenticity and integrity. - Web of Trust: PGP uses user-signed keys and trust levels rather than a central CA.

THEORY:	Pretty Good Privacy (PGP) combines symmetric encryption and asymmetric (public-key) cryptography to secure communications. 1. **Encryption**: The sender uses the recipient's public key to encrypt a message. The recipient decrypts it using their private key. 2. **Signing**: The sender uses their private key to sign a message, and the recipient verifies the signature using the sender's public key. 3. **Web of Trust**: PGP uses a decentralized trust model where users sign each other's keys to build trust. 4. **Revocation**: Revocation certificates are used to invalidate keys if they are compromised. 5. **Confidentiality, Integrity, Authentication, and Non-repudiation**: are achieved using a mix of RSA and AES algorithms.

Implementation:	Step 0 – Install GnuPG:
	
	sudo apt update
	sudo apt install gnupg
	
	`students@students-HP-280-G3-MT:~$ sudo apt install gnupg`
	`Reading package lists... Done`
	`Building dependency tree`
	`Reading state information... Done`
	`gnupg is already the newest version (2.2.19-3ubuntu2.5).`
	`gnupg set to manually installed.`
	`The following packages were automatically installed and are no longer required:`
	`  gir1.2-goa-1.0 libfwupdplugin1 libxmlb1`
	`Use 'sudo apt autoremove' to remove them.`
	`0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.`
	
	Step 1 – Generate Key Pair for Alice and amanda:
	
	gpg --full-generate-key
	[List and verify keys]
	
	For Alice:

```
students@students-HP-280-G3-MT:~$ gpg --full-generate-key
gpg (GnuPG) 2.2.19; Copyright (C) 2019 Free Software Foundation, Inc.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Please select what kind of key you want:
   (1) RSA and RSA (default)
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
  (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 1024
Requested keysize is 1024 bits
Please specify how long the key should be valid.
         0 = key does not expire
      <n>  = key expires in n days
      <n>w = key expires in n weeks
      <n>m = key expires in n months
      <n>y = key expires in n years
Key is valid for? (0) 0
Key does not expire at all
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: alice
Email address: alice@example.com
Comment: hello
You selected this USER-ID:
    "alice (hello) <alice@example.com>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perfor
m
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
```

	We need to generate a lot of random bytes. It is a good idea to perform some other action (type on the keyboard, move the mouse, utilize the disks) during the prime generation; this gives the random number generator a better chance to gain enough entropy. gpg: key 18FF10FD8ACD2B2B marked as ultimately trusted gpg: directory '/home/students/.gnupg/openpgp-revocs.d' created gpg: revocation certificate stored as '/home/students/.gnupg/openpgp-revocs.d/99B1BB9828F7A50838B0D1B318FF10FD8ACD2B2B.rev' public and secret key created and signed. pub rsa1024 2025-10-16 [SC] 99B1BB9828F7A50838B0D1B318FF10FD8ACD2B2B uid alice (hello) <alice@example.com> sub rsa1024 2025-10-16 [E] For Amanda:

```
students@students-HP-280-G3-MT:~$ gpg --full-generate-key
gpg (GnuPG) 2.2.19; Copyright (C) 2019 Free Software Foundation, Inc.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Please select what kind of key you want:
   (1) RSA and RSA (default)
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
  (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 1024
Requested keysize is 1024 bits
Please specify how long the key should be valid.
         0 = key does not expire
      <n>  = key expires in n days
      <n>w = key expires in n weeks
      <n>m = key expires in n months
      <n>y = key expires in n years
Key is valid for? (0) 0
Key does not expire at all
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: bob
Name must be at least 5 characters long
Real name: amanda
Email address: amanda@example.com
Comment: hello
You selected this USER-ID:
    "amanda (hello) <amanda@example.com>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
```

```
We need to generate a lot of random bytes. It is a good idea to perfor
m
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perfor
m
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: key F4002FEF2CC5596C marked as ultimately trusted
gpg: revocation certificate stored as '/home/students/.gnupg/openpgp-r
evocs.d/8B211FD1CCC1AA858D4961F8F4002FEF2CC5596C.rev'
public and secret key created and signed.

pub   rsa1024 2025-10-16 [SC]
      8B211FD1CCC1AA858D4961F8F4002FEF2CC5596C
uid                      amanda (hello) <amanda@example.com>
sub   rsa1024 2025-10-16 [E]
```

gpg --list-keys:

gpg --list-secret-keys:

```
students@students-HP-280-G3-MT:~$ gpg --list-keys
gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: depth: 0  valid:   2  signed:   0  trust: 0-, 0q, 0n, 0m, 0f, 2u
/home/students/.gnupg/pubring.kbx
--------------------------------
pub   rsa1024 2025-10-16 [SC]
      99B1BB9828F7A50838B0D1B318FF10FD8ACD2B2B
uid           [ultimate] alice (hello) <alice@example.com>
sub   rsa1024 2025-10-16 [E]

pub   rsa1024 2025-10-16 [SC]
      8B211FD1CCC1AA858D4961F8F4002FEF2CC5596C
uid           [ultimate] amanda (hello) <amanda@example.com>
sub   rsa1024 2025-10-16 [E]

students@students-HP-280-G3-MT:~$ gpg --list-secret-keys
/home/students/.gnupg/pubring.kbx
--------------------------------
sec   rsa1024 2025-10-16 [SC]
      99B1BB9828F7A50838B0D1B318FF10FD8ACD2B2B
uid           [ultimate] alice (hello) <alice@example.com>
ssb   rsa1024 2025-10-16 [E]

sec   rsa1024 2025-10-16 [SC]
      8B211FD1CCC1AA858D4961F8F4002FEF2CC5596C
uid           [ultimate] amanda (hello) <amanda@example.com>
ssb   rsa1024 2025-10-16 [E]
```

Step 2 – Create Revocation Certificates:

gpg --output alice-revoke.asc --gen-revoke alice@example.com

```
students@students-HP-280-G3-MT:~$ gpg --output alice-revoke.asc --gen-
revoke alice@example.com

sec  rsa1024/18FF10FD8ACD2B2B 2025-10-16 alice (hello) <alice@example.
com>

Create a revocation certificate for this key? (y/N) y
Please select the reason for the revocation:
  0 = No reason specified
  1 = Key has been compromised
  2 = Key is superseded
  3 = Key is no longer used
  Q = Cancel
(Probably you want to select 1 here)
Your decision? 0
Enter an optional description; end it with an empty line:
>
Reason for revocation: No reason specified
(No description given)
Is this okay? (y/N) y
ASCII armored output forced.
Revocation certificate created.

Please move it to a medium which you can hide away; if Mallory gets
access to this certificate he can use it to make your key unusable.
It is smart to print this certificate and store it away, just in case
your media become unreadable.  But have some caution:  The print syste
m of
your machine might store the data and make it available to others!
```

gpg --output amanda-revoke.asc --gen-revoke amanda@example.com

```
students@students-HP-280-G3-MT:~$ gpg --output amanda-revoke.asc --gen
-revoke amanda@example.com

sec  rsa1024/F4002FEF2CC5596C 2025-10-16 amanda (hello) <amanda@exampl
e.com>

Create a revocation certificate for this key? (y/N) y
Please select the reason for the revocation:
  0 = No reason specified
  1 = Key has been compromised
  2 = Key is superseded
  3 = Key is no longer used
  Q = Cancel
(Probably you want to select 1 here)
Your decision? 0
Enter an optional description; end it with an empty line:
>
Reason for revocation: No reason specified
(No description given)
Is this okay? (y/N) y
ASCII armored output forced.
Revocation certificate created.

Please move it to a medium which you can hide away; if Mallory gets
access to this certificate he can use it to make your key unusable.
It is smart to print this certificate and store it away, just in case
your media become unreadable.  But have some caution:  The print syste
m of
your machine might store the data and make it available to others!
```

Step 3 – Export Public Keys:

gpg --armor --export alice@example.com > alice_pub.asc
gpg --armor --export amanda@example.com > amanda_pub.asc

```
students@students-HP-280-G3-MT:~$ gpg --armor --export alice@example.c
om > alice_pub.asc
students@students-HP-280-G3-MT:~$ gpg --armor --export amanda@example.
com > amanda_pub.asc
students@students-HP-280-G3-MT:~$ cat alice_pub.asc
-----BEGIN PGP PUBLIC KEY BLOCK-----

mI0EaPC58wEEANUVncJ6VXHiYwB4jNapqAhEs4Ns05dkjFlv3BVtSBsOhJE0BQ6M
jrtxKsSJPfrrI5D8H7YoZ+pKDitcGae3eaf0tUKB0F7RZdlNQzx05kVEG7nw9D6l
Y3RJBqzMRW9Vvex6D9KcRJ6J+u/UrkUPG6SlfoMlWvQqV4u9a4w6ARonABEBAAG0
IWFsaWNlIChoZWxsbykgPGFsaWNlQGV4YW1wbGUuY29tPojOBBMBCgA4FiEEmbG7
mCj3pQg4sNGzGP8Q/YrNKysFAmjwufMCGwMFCwkIBwIGFQoJCAsCBBYCAwECHgEC
F4AACgkQGP8Q/YrNKyu56AP/arswRz14/R5jJHILaByZPpVac+42u+BWPbER/b58
vzv2GYC6OTH+OQ2PzwDUaVbafkCWMiG2PRyNe4jfdBs4KQmOuP9Xp2sgjY+h9XZZ
0dkG63cPiFGpcSYvTYBHzVuYGuzfKLBgfBuWB/tqIZk8NzwtJygyYWd1PO0hiSUg
wSK4jQRo8LnzAQQAtNJH9sM3bky2fdbzh161nvr1YSIYbkgPzv1AXpjvtRakTB2E
2XnxLtZoR8f9Tqm6G0PawFobrlCp+QURkaqNu3GVryz9nxboPWqwgFq2us9imJI8
kliFmez079Wr0/pEWn7zDK8LizcHFamM0fBWfzUZSN8FazxrhnYEEHHe8TEAEQEA
AYi2BBgBCgAgFiEEmbG7mCj3pQg4sNGzGP8Q/YrNKysFAmjwufMCGwwACgkQGP8Q
/YrNKys06wQAwqKnZmCeuyIwIna/Wfp7sdio3I4ap3sO4cALuCWc089WsN9OKsvC
eYlD5DYx+ZGoQarjnCi8na9jpRqYpKgmH8dYv2m2W1nbK+39KN23pq3AQTzdC3C5
mRFM4a+vzpKB5FbffOIpc5XAJfRmfotrmuKcx+bqnCf7UOkt7Xe0b6o=
=SmuE
-----END PGP PUBLIC KEY BLOCK-----
```

```
students@students-HP-280-G3-MT:~$ cat amanda_pub.asc
-----BEGIN PGP PUBLIC KEY BLOCK-----

mI0EaPC6cAEEAMlg2lVcb++kmAXCdjFppmJk9Cxuzgys+Weo3avMzH6Ymex8bSdX
J7qhn2Khbb6mhu+YFHtmC2xj0G8BeYib2aj6Bj2WcsWglsok5v1/V3smUxW1A4Xm
iFOuRsBV26JE6oGx8nP4wGPAV+tW3YxrmGqw9AqN77Epl7Mx4omf8/vRABEBAAG0
I2FtYW5kYSAoaGVsbG8pIDxhbWFuZGFAZXhhbXBsZS5jb20+iM4EEwEKADgWIQSL
IR/RzMGqhY1JYfj0AC/vLMVZbAUCaPC6cAIbAwULCQgHAgYVCgkICwIEFgIDAQIe
AQIXgAAKCRD0AC/vLMVZbFVgA/9xWTsJjNHRkpCtb8tQLPkpN6gU0z3aQoMudM++
7hb2fryfzXBqLFfYFW/UKpjs5SZfiQw1V5XSBmzfC3NBMYeINfaP9h9DWaSWbopN
goySX6H8NCeAo+r5uKFy4oJ5S3IOk9OVB3d5O/OKJ/VO1lP9wQfi1u3P9J0OkjHU
jAR7r7iNBGjwunABBADPjfgE+9bxunxv/S94osw+QVhZ94SWPdXnnkApep5AM4UD
mvyJ7zF8TjwR9uALuWdWITclC+nX8pPXwU3KFdZuzkAVu2W7NOlh2eOkSjAOC81h
en2Baw94GmaPumRtyz8Z/aSx1VCQEHmckK+6PxovBioiMcU7a1x/14synL6NOQAR
AQABiLYEGAEKACAWIQSLIR/RzMGqhY1JYfj0AC/vLMVZbAUCaPC6cAIbDAAKCRD0
AC/vLMVZbOcAA/93AXTE3UuUVij+IksjwnNrCogBSDhLz+Z/yi4/VuEOdHAGahha
0/ZVFPHRgFedEKvfNHNuz/Hud7PpY5hXvbzQwiPAWPZoDlB4DM3rTZk0QfZTUXFw
T5Rp67g7RY0ikaxihhsSq4gaNStKKYlekZz6owtl/h6V1+LFlai0hCIo6Q==
=qQQS
-----END PGP PUBLIC KEY BLOCK-----
```

Step 4 – Import Other Person’s Public Key:

amanda imports Alice's key:
gpg --import alice_pub.asc

```
students@students-HP-280-G3-MT:~$ gpg --import alice_pub.asc
gpg: key 18FF10FD8ACD2B2B: "alice (hello) <alice@example.com>" not cha
nged
gpg: Total number processed: 1
gpg:               unchanged: 1
```

Step 5 – Encrypt a File (amanda to Alice):

echo 'Hello Alice, this is a secret message.' > message.txt
gpg --encrypt --recipient alice@example.com --armor -o message_to_alice.asc message.txt

```
students@students-HP-280-G3-MT:~$ echo 'Hello Alice, this is a secret
message.' > message.txt
students@students-HP-280-G3-MT:~$ gpg --encrypt --recipient alice@exam
ple.com --armor -o message_to_alice.asc message.txt
```

Step 6 – Decrypt the File (Alice):

gpg --output decrypted.txt --decrypt message_to_alice.asc
cat decrypted.txt

```
students@students-HP-280-G3-MT:~$ gpg --output decrypted.txt --decrypt
 message_to_alice.asc
gpg: encrypted with 1024-bit RSA key, ID 43A631E6DB7194B0, created 202
5-10-16
      "alice (hello) <alice@example.com>"
students@students-HP-280-G3-MT:~$ cat decrypted.txt
Hello Alice, this is a secret message.
```

Step 7 – Create Digital Signature (Alice):

gpg --output report.sig --detach-sign report.pdf
gpg --clearsign message.txt

```
students@students-HP-280-G3-MT:~$ echo "This is a sample report." > re
port.txt
students@students-HP-280-G3-MT:~$ gpg --output report.sig --detach-sig
n report.pdf
gpg: can't open 'report.pdf': No such file or directory
gpg: signing failed: No such file or directory
students@students-HP-280-G3-MT:~$ libreoffice --headless --convert-to
pdf report.txt
students@students-HP-280-G3-MT:~$ gpg --output report.sig --detach-sig
n report.pdf
students@students-HP-280-G3-MT:~$ gpg --clearsign message.txt
```

Step 8 – Verify Signatures :

gpg --verify report.sig report.pdf

```
students@students-HP-280-G3-MT:~$ gpg --verify report.sig report.pdf
gpg: Signature made Thursday 16 October 2025 03:07:40 PM IST
gpg:                using RSA key 99B1BB9828F7A50838B0D1B318FF10FD8ACD
2B2B
gpg: Good signature from "alice (hello) <alice@example.com>" [ultimate
]
```

Step 9 – Encrypt + Sign (Alice to amanda):

gpg --encrypt --sign --recipient amanda@example.com --armor -o message_to_amanda.asc message.txt

gpg --output decrypted.txt --decrypt message_to_amanda.asc

```
students@students-HP-280-G3-MT:~$ gpg --encrypt --sign --recipient ama
nda@example.com --armor -o message_to_amanda.asc message.txt
students@students-HP-280-G3-MT:~$ gpg --output decrypted.txt --decrypt
 message_to_amanda.asc
gpg: encrypted with 1024-bit RSA key, ID 6E28AE94E1162076, created 202
5-10-16
      "amanda (hello) <amanda@example.com>"
File 'decrypted.txt' exists. Overwrite? (y/N) y
gpg: Signature made Thursday 16 October 2025 03:12:46 PM IST
gpg:                using RSA key 99B1BB9828F7A50838B0D1B318FF10FD8ACD
2B2B
gpg: Good signature from "alice (hello) <alice@example.com>" [ultimate
]
students@students-HP-280-G3-MT:~$ cat decrypted.txt
Hello Alice, this is a secret message.
```

Step 10 – Demonstrate Failed Verification:

gpg --output tampered.sig --detach-sign message.txt
echo 'Extra line' >> message.txt
gpg --verify tampered.sig message.txt

```
students@students-HP-280-G3-MT:~$ gpg --output tampered.sig --detach-s
ign message.txt
students@students-HP-280-G3-MT:~$ echo 'Extra line' >> message.txt
students@students-HP-280-G3-MT:~$ gpg --verify tampered.sig message.tx
t
gpg: Signature made Thursday 16 October 2025 03:14:37 PM IST
gpg:                using RSA key 99B1BB9828F7A50838B0D1B318FF10FD8ACD
2B2B
gpg: BAD signature from "alice (hello) <alice@example.com>" [ultimate]
```

Step 11 – Key Trust and Revocation:

gpg --sign-key amanda@example.com

```
students@students-HP-280-G3-MT:~$ gpg --sign-key amanda@example.com

sec  rsa1024/F4002FEF2CC5596C
     created: 2025-10-16  expires: never       usage: SC
     trust: ultimate      validity: ultimate
ssb  rsa1024/6E28AE94E1162076
     created: 2025-10-16  expires: never       usage: E
[ultimate] (1). amanda (hello) <amanda@example.com>


sec  rsa1024/F4002FEF2CC5596C
     created: 2025-10-16  expires: never       usage: SC
     trust: ultimate      validity: ultimate
 Primary key fingerprint: 8B21 1FD1 CCC1 AA85 8D49  61F8 F400 2FEF 2CC
5 596C

     amanda (hello) <amanda@example.com>

Are you sure that you want to sign this key with your
key "alice (hello) <alice@example.com>" (18FF10FD8ACD2B2B)

Really sign? (y/N) y
```

gpg --edit-key amanda@example.com

#trust #quit

```
students@students-HP-280-G3-MT:~$ gpg --edit-key amanda@example.com
gpg (GnuPG) 2.2.19; Copyright (C) 2019 Free Software Foundation, Inc.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Secret key is available.

gpg: checking the trustdb
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: depth: 0  valid:   2  signed:   0  trust: 0-, 0q, 0n, 0m, 0f, 2u
sec  rsa1024/F4002FEF2CC5596C
     created: 2025-10-16  expires: never       usage: SC
     trust: ultimate      validity: ultimate
ssb  rsa1024/6E28AE94E1162076
     created: 2025-10-16  expires: never       usage: E
[ultimate] (1). amanda (hello) <amanda@example.com>

gpg> # trust
gpg> trust
sec  rsa1024/F4002FEF2CC5596C
     created: 2025-10-16  expires: never       usage: SC
     trust: ultimate      validity: ultimate
ssb  rsa1024/6E28AE94E1162076
     created: 2025-10-16  expires: never       usage: E
[ultimate] (1). amanda (hello) <amanda@example.com>

Please decide how far you trust this user to correctly verify other us
ers' keys
(by looking at passports, checking fingerprints from different sources
, etc.)

  1 = I don't know or won't say
  2 = I do NOT trust
  3 = I trust marginally
  4 = I trust fully
  5 = I trust ultimately
  m = back to the main menu
```

```
Your decision? 5
Do you really want to set this key to ultimate trust? (y/N) y

sec  rsa1024/F4002FEF2CC5596C
     created: 2025-10-16  expires: never       usage: SC
     trust: ultimate      validity: ultimate
ssb  rsa1024/6E28AE94E1162076
     created: 2025-10-16  expires: never       usage: E
[ultimate] (1). amanda (hello) <amanda@example.com>

gpg> quit
```

	gpg --import alice-revoke.asc 
Conclusion:	In this experiment, Pretty Good Privacy (PGP) was implemented using GnuPG. We successfully demonstrated key generation, message encryption and decryption, digital signature creation and verification, key export/import, and revocation. The experiment highlights how PGP ensures confidentiality, integrity, authenticity, and non-repudiation in secure communication.